

Module 3 — Customization

Lesson 3.1

CLAUDE.md — project memory done right

CLAUDE.md is auto-loaded at session start. Use it for project facts Claude couldn't otherwise know.

Why it matters

Claude reads `CLAUDE.md` from your repo (and `~/.claude/CLAUDE.md` for your user-global preferences) at the start of every session. Whatever you put there becomes free, always-on context. A well-tuned `CLAUDE.md` is the difference between Claude needing 20 questions to start a task vs. starting cleanly.

Where files are loaded from

In order:

- 1 `~/.claude/CLAUDE.md` — user-global (your preferences, applies to every project)
- 2 `<project>/CLAUDE.md` — project-wide (committed; shared with the team)
- 3 `<project>/CLAUDE.local.md` — project-local (gitignored; your private notes for this repo)
- 4 Nested `CLAUDE.md` in subdirectories — auto-loaded when Claude works in that subtree

The nested-file trick is underused. A monorepo can have a service-specific `CLAUDE.md` per service that loads only when Claude is operating in that subtree.

What to put in CLAUDE.md

Good content is **non-derivable** — things Claude can't figure out by reading the code.

Category	Examples
Conventions	"Use snake_case for DB columns, camelCase in API responses."
Commands	"Run tests with <code>pnpm test:unit</code> . Don't use <code>npm</code> ; we're on <code>pnpm</code> ."
Architectural decisions	"The billing subsystem uses event sourcing — never mutate <code>BillingEvent</code> rows."
Domain language	"An 'account' is a billing entity; a 'user' is a login. They're not 1:1."
Gotchas	"The CI flakes on <code>tests/integration/test_email.py</code> — re-run before debugging."
Tooling	"Type-check with <code>pnpm typecheck</code> . Lint with <code>pnpm lint --fix</code> ."

What NOT to put

- **Anything obvious from the code.** Don't document the directory structure — Claude can `ls`.
- **Anything that changes constantly.** Active sprint goals belong in a tracker, not here.
- **Long-form documentation.** `CLAUDE.md` is a cheat sheet, not a wiki.
- **Secrets.** Ever. Use env vars and settings instead.
- **Style preferences your linter already enforces.** Trust the linter.

Template

```
# Project: <name>

## What this is
<one-paragraph description – what the system does, who it's for>

## Commands
- Install: `pnpm install`
- Dev: `pnpm dev`
- Test: `pnpm test`
- Typecheck: `pnpm typecheck`
- Lint: `pnpm lint`

## Conventions
- DB columns: snake_case. API fields: camelCase. Conversion happens in
  `src/api/serializers.ts`.
- Errors: throw `AppError` subclasses, never bare `Error`.
- Tests live next to source as `*.test.ts`.

## Architecture notes
- Auth uses JWT in HttpOnly cookies, refreshed by `/auth/refresh`.
- The job runner is BullMQ on Redis; jobs live in `src/jobs/`.

## Gotchas
- `tests/integration/email.test.ts` is flaky on CI; not a real regression usually.
- Don't run `pnpm db:reset` against staging – it really does drop the DB.

## When in doubt
- Ask before adding a new dependency.
- Prefer modifying existing tests over adding parallel ones.
```

Aim for 50–200 lines. Beyond that you're writing documentation.

How `/init` fits in

`/init` (covered in 1.3) generates a starter `CLAUDE.md` based on what Claude infers from the repo. Treat its output as a draft, then prune ruthlessly.

Updating `CLAUDE.md` from inside Claude Code

When Claude learns something the team will benefit from, ask it to update:

"Add a line to `CLAUDE.md` under Gotchas: 'Webpack config must be edited in `webpack.config.cjs`, not the `.mjs` shim — the shim is just for Vite compat.' Commit it."

This turns one-off discoveries into permanent team knowledge.

Per-team patterns

For shared repos, agree on:

- Who owns `CLAUDE.md` updates (the team, not random PRs).
- What level of detail belongs there vs. in real docs.

- Whether `.local.md` is encouraged (it is — let people customize without polluting shared scope).

Try it

- 1 Run `/init` in a project that doesn't have a `CLAUDE.md`.
- 2 Trim what it produced to under 100 lines. Keep only non-derivable facts.
- 3 Add three "gotchas" you know about the repo that aren't in the code.
- 4 Start a fresh session and ask Claude an ambiguous question. Note whether it leans on the `CLAUDE.md` context.

Gotchas

- **Don't dump style guides.** The linter is the source of truth; Claude will read the linter config if needed.
- **Avoid huge tables of file paths.** They go stale fast. Better: "look in `src/api/`."
- **Don't repeat what the README already says.** Link to it: "See `README.md` for setup."
- **CLAUDE.md changes need a session restart** to fully reload, though many take effect mid-session via the system reminder system.

Further reading

- 3.2 `settings.json` — for permissions/env, not for memory
- Memory system docs: <https://docs.claude.com/en/docs/claude-code/memory>

Next

→ 3.2 `settings.json` — permissions, env, and scope